

(the)

UPSET

QUARTET

Ivan, Tahmid, Tyler and Ethan click here!

Project Manager: Ivan

TNPG: Upset Quartet

SoftDev

P00: Full-Quick {PIVOT}

TARGET SHIP DATE: 2026-11-20 (TBD)

Time Spent: 4 hrs

List of Program Components

and Site Map because both were done at the same time!

- There are the necessary and optional pages for this project to work.
    - login.html - This is the default page that the user is redirected to if they are not logged in. The user should not be able to access any site except for this and signup.html if they are not logged in. login.html prompts the user to input their username and password, then redirects the user to /user/[username].html if successfully logged in.
    - signup.html - This is the page where you sign up for your account, only accessible to non logged in users. signup.html prompts the user to input a new username and the same password twice, then redirects the user to /user/[username].html if successfully signed up. Please note that because the profile page is empty, there should be randomized stories “recommended” to the user. (It’s better to look out than no stories.)
    - /user/[username].html - This is a profile page that each user should be able to see. It will display the user’s username as well as the stories that the user has contributed to. If it is a new user, 4 randomized (suggested) stories are placed, though this may also apply to any user at all. Temporarily, users should not be able to view other users’ profile pages, but this may be implemented if we have enough time.
    - /page/[story].html - Every story that a user reads should be in this format. The user can choose to edit the story, but keep in mind that the user cannot go to any

other page once they decide to start editing the story. There could possibly be a confirmation action to ask whether the user wants to edit the story.

- edit.html - Page to create a story, just an edit page with empty fields. After saving, create paths with the story name in /page and /edit to view and edit the story.
- /edit/[story].html - When editing a story, the user can only see what the previous person added. This is in contrast to when reading the story, where the user can see the entire story. Regardless if the user *cancels* or *updates* the story successfully, once the user stops editing the story, they cannot edit the story afterwards. Furthermore, while the user has a session in editing the story, they cannot go back to any other page, even logging out. The user must confirm to cancel or update the story if they want to do so.
- index.html - Page that displays all of the stories on the site and clicking on a hyperlink redirects the user to the reading page for that story.

- There are other boring things, but it's pretty self-evident. There needs to be a Flask server running — because of our limited knowledge — in order to get the website up. Unlike concatenating user information into an easily accessible text file, it's now stored into a SQLite3 table. And finally, Jinja for your favorite usage of {{ variable }}.

Possible Routing:

Logged-Out User

login.html -> signup.html -> /user/[username].html

or login.html -> login.html -> /user/[username].html

Logged-In User

/user/[username].html -> index.html -> /page/[story].html -> edit/[page].html (edit story) -> /page/[story].html -> /user/[username].html -> edit.html (create story) -> /page/[story].html -> /user/[username].html -> logout.html (logs out) -> login.html

Component Relationships

Also known as the client-server interactions… This'll be tough.

It'll be easier to explain through different scenarios: what will happen at each step?

Scenario I: The user creates a web page

1) On the local client, the user hits the submit button. This sends a POST request to the Flask web server.
2) Flask requests the necessary information from a SQLite3 table, like page title. However, since the user is creating a page, this step isn't needed unlike when the user is updating the page.
3) The table should return some data back to Flask. Thereafter, it would render templates using Jinja, compile the HTML, attach the necessary CSS and zip that up ready for the user to read.

Keep in mind that this is an ultra-simplified version of what goes on. Again, Robert"); DROP TABLE Students;--. There's not enough room here, go down to view the component diagram.

Scenario II: The user creates an account.

1) On the local client, the user hits the submit button. This sends a POST request to the Flask web server.
2) Flask checks the client's information, primarily by their cookie, to see if they are logged in. If so, then Flask tries to redirect the user to their profile page. Otherwise, Flask requests the necessary information from a SQLite3 table, to further check if the account creation is valid. If the username is already taken, a field isn't filled out, or there are no matching passwords, then an error message is displayed. Then, the Flask server should append the user information to the necessary table.
3) Once the information has been safely stored, Flask then needs to create a cookie, logging the user into their new account. The user should be redirected to the main page, happily logged in.

If two people try to create an account at the same time with the same username, first Flask/Python checks if it exists, then have an SQLite3 function check if it exists, if it does, then an error should be thrown. Else, the account should be created to the lucky user who created it "first".

Scenario III: The user adds onto an existing story.

For the sake of simplicity, a note about Flask/SQLite3 tables will merely be written instead of restating the same thing over and over again. This time, Flask needs to read the data associated with the current page and check if the user editing has added onto the story before or not. If not, the user can then add onto the story and their username will be added to a list in Flask attached to the story and the same user can no longer add to the story. If the user's name is already on the list, then they can only view the story.

Added content will be sanitized of any SQLite3 keywords before being appended to the story content.

Scenario IV: The user searches for every page in the index.

Beyond reading every page in the SQLite3 table and storing it in a list to be displayed, if the user tries to search for a certain character ("a"), what results should be listed foremost? Should there be customizable ways to sort the pages you read?

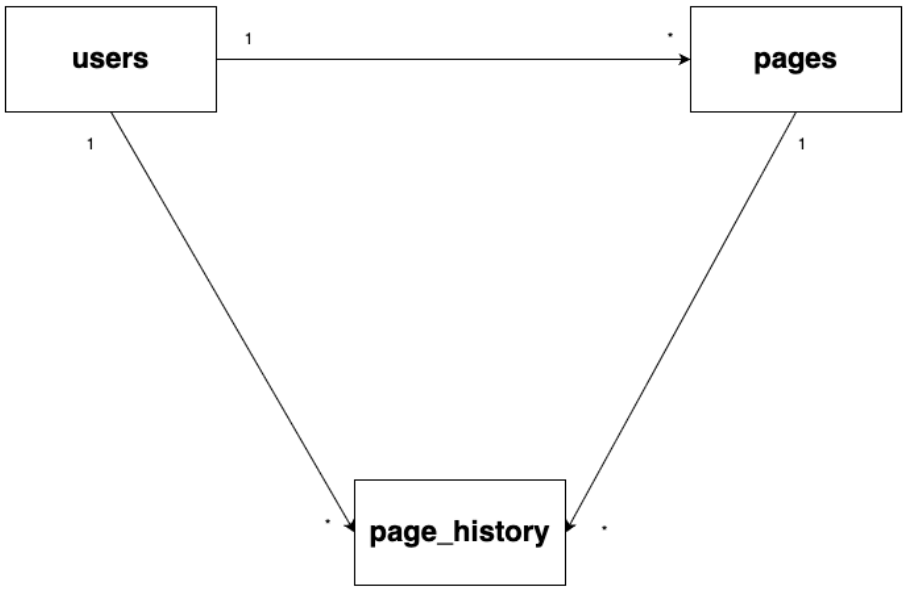
In summary, most of the actions that take place involve five key steps.

1. The user requests an operation to be handled.
2. Flask confirms that the user has the capability to do so, and if necessary, in multiple ways.
3. Flask requests/modifies/appends/deletes information from the SQLite3 table.
4. Flask modifies/redirects/creates a web page with the necessary information for the viewer.
5. The user is able to see the changes made once the operation has been completed.

Database Organization

There needs to be three tables that are necessary for the database. The first table necessary is the users table, that has the columns id: integer (technically each username is unique, but the id is a stable way of verifying the authenticity and uniqueness of each user; this can also be used in place of the username as the appendix of a link), username: String, password: String. (A stretch goal of ours is to have a private/public key setup where the password is encrypted in base64). The second table necessary is the pages table, where each individual page is stored with the raw content. It should have an id: integer (PK), title: String, text: String, creator_id: integer, and created_at: integer. The third table should run in conjunction with the pages table and is the page_history table. The third table will contain the edits of all other pages, linked by the respective id PK of each page, which contains an edit_id: integer (PK) (identifier for the edit), the last_editor_id, last_updates time stamp as well as the text that was added, and id: integer (non-PK). The full story will be an append of the original story and an iteration through all the changes and if someone wants to add onto a story, they will only see the string from the last edit.

To prevent multiple people from pushing edits at the same time, compare the original edit_id to the current edit_id. If they are the same, nobody pushed changes in the time the edit was made. If they are not the same, then the text displayed being added onto must be updated before anyone else can push their edits to prevent edit conflicts.



A simple diagram showing the cardinality of each table.

Please do not fret over whether the symbol should be an asterisk or the letter N.

Security

To prevent people from doing naughty things to ruin our database, every input from the user must be sanitized. Queries are parameterised to have SQLite treat the input as pure data rather than code, instead of using Python f-strings.
So rather than having Flask doing this:

```
cursor.execute(f"SELECT * FROM users WHERE name = '{user_input}'")
```

Where the entire statement being executed will be treated as valid SQL code, meaning that Flask won't be able to know what's server-side SQL code and what's user-submitted SQL code.

We are going to do this:

```
cursor.execute("SELECT * FROM users WHERE name = ?", (user_input,))
```

Where SQLite takes the SQL statement and compiles it separately and only uses the user_input at execution time, and thus can never be parsed as SQL code.

Just a small note, but the SECRET_KEY should be a randomly generated hex token seeded at the time of the start of server runtime.

Task Breakdown and Assignments of Each Task

Person 1: Lin, Tyler (backup: Uddin, Tahmid)
SQLite3 Database Architect
Responsible for constructing the structure of the database as well as managing it.

Person 2: Uddin, Tahmid (backup: Lin, Tyler)
Authentication and Session Master | SQLite3 Database Assistant
Responsible for ensuring the integrity of the database, as well as helping the Database Architect.

Person 3: Chan, Ivan (backup: Chang, Ethan)
Story Engine and Page Handler

Write the main engine for Flask/Python/Jinja setup and help jumpstart the code for SQLite3 tables and design for each page.

Person 4: Chang, Ethan (backup: Chan, Ivan)

Documentation and Front-End Developer

Make sure the repository is up-to-date and work on CSS/front end development.

Component Relationship Diagram and Routing Diagram

Totally not the same diagram as last time.

